A PROJECT REPORT

ON

E-LEARNING QUIZ COMPETITION AND LEADERBOARD SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Student Name	RollNumber
Kumari Shristi	23BCS10551
Kunal Kumar	23BCS12111
Manmohan Kumar	23BCS12163

Under the Guidance of

Er. Rahul Durgapal

Assistant Professor, Department of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Chandigarh University

Mohali, Panjab – 140301

April, 2026

BONAFIDE CERTIFICATE

This is to certify that this project report titled **"E-Learning Quiz Competition and Leaderboard System"** is the bonafide work of **Kumari Shristi (23BCS10551)**, **Kunal Kumar (23BCS12111)**, and **Manmohan Kumar (23BCS12163)**, who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SUPERVISOR	**HEAD OF DEPARTMENT**
Er. Rahul Durgapal	Prof. Dr. Sanjeev Kumar
Technical Trainer	Professor and Head
Department of CSE	Department of CSE
Chandigarh University	Chandigarh University

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to all those who have helped us in completing this project successfully.

We are deeply indebted to our project supervisor, **Er. Rahul Durgapal**, As-sistant Professor, Department of Computer Science and Engineering, for the valuable guidance, continuous encouragement, and constructive suggestions provided throughout the development of this project. Their expertise and mentorship were instrumental in shaping the direction of this work.

We sincerely thank **Prof.** Dr. Sanjeev Kumar, Head of the Department of Computer Science and Engineering, for providing the necessary resources and a conducive academic environment.

We are also grateful to the **Principal**, Chandigarh University, for the constant support and encouragement extended to us.

Our heartfelt thanks go to all the **faculty members** of the Department of Computer Science and Engineering for their academic support and for sharing their knowledge throughout the course of our study.

We sincerely thank the **technical staff** of the Computer Science laboratories for their assistance and cooperation during the practical implementation phase of the project.

Finally, we owe a debt of gratitude to our **parents and family members** for their moral support, patience, and encouragement, and to our **friends and classmates** whose motivation helped us overcome the challenges encountered during this project.

Kumari Shristi	**23BCS10551**
Kunal Kumar	**23BCS12111**
Manmohan Kumar	**23BCS12163**

Contents

1 Introduction 11

1.1 Client Identification and Need 11

1.2 Problem Statement 11

1.3 Objectives 11

1.4 Scope of the Project 12

1.5 Timeline 13

1.6 Organisation of the Report 13

2 Literature Survey 14

2.1 Overview of E-Learning Systems 14

2.2 Existing Systems and Research Trends 14

2.2.1 Online Quiz Platforms 14

2.2.2 Research on Gamification in Education 15

2.2.3 Web Technologies for E-Learning 15

2.3 Limitations of Existing Systems 15

2.4 Problem Definition 15

2.5 Goals and Objectives Derived from Literature 16

3 Design and Development 17

3.1 System Architecture 17

3.2 Technologies Used 18

3.3 Alternative Designs Considered 18

3.3.1 Design Alternative 1 – PHP + MySQL 18

3.3.2 Design Alternative 2 – Java Servlets + JSP + JDBC + MySQL (Selected) 18

3.3.3 Design Comparison 19

3.4 System Flowchart 20

3.5 Database Design 20

3.5.1 Entity-Relationship Diagram Description 20

3.5.2 SQL Table Definitions 21
3.6 Module Descriptions 23
3.6.1 Admin Module 23
3.6.2 User / Student Module 23
3.6.3 Quiz Engine 23
3.6.4 Leaderboard Module 24
3.7 Pseudo Code for Core Logic 24
3.8 Design Constraints 25
3.8.1 Economic Constraints 25
3.8.2 Ethical Constraints 25
3.8.3 Safety and Security Constraints 25
3.8.4 Social and Environmental Constraints 26

4 Results and Validation 27
4.1 Implementation Overview 27
4.2 Screenshot Descriptions 27
4.2.1 Login and Registration Page 27
4.2.2 Admin Dashboard 27
4.2.3 Quiz Attempt Page 28
4.2.4 Result Page 28
4.2.5 Leaderboard Page 29
4.3 Testing 29
4.3.1 Unit Testing 29
4.3.2 Integration Testing 30
4.3.3 System Testing 30
4.4 Performance Analysis 30

5 Conclusion and Future Work 32
5.1 Summary 32
5.2 Limitations 32
5.3 Future Enhancements 33

A User Manual 35
A.1 System Requirements 35
A.2 Installation and Deployment Steps 35
A.3 User Guide – Student 36

A.4 User Guide – Administrator 36

B Achievements **37**

List of Figures

1 Graphical Abstract – System Architecture Overview 9

3.1 Three-Tier MVC Architecture 17

3.2 System Flowchart – User and Admin Flow 20

4.1 User Login Page 27

4.2 Administrator Dashboard 28

4.3 Quiz Attempt Interface with Countdown Timer 28

4.4 Quiz Result Page 29

4.5 Performance Analysis – Average Response Times 31

List of Tables

1 List of Abbreviations and Symbols 10

1.1 Project Development Timeline 13

2.1 Limitations of Existing Systems 15

3.1 Technologies Used in the Project 18
3.2 Design Alternative Comparison 19

4.1 Unit Test Results 29
4.2 Integration Test Results 30
4.3 Performance Test Results 30

ABSTRACT

The rapid advancement of digital technologies has transformed the domain of education, giving rise to e-learning platforms that offer interactive, engaging, and scalable learning experiences. This project presents the design and implementation of an **E-Learning Quiz Competition and Leaderboard System** – a web-based application built using Java Servlets, JavaServer Pages (JSP), JDBC, MySQL, HTML, and CSS, adhering to Object-Oriented Programming (OOP) principles.

The system enables educational institutions and online learning environments to conduct timed quiz competitions among registered users. The platform is structured around two primary roles: the **Administrator** and the **Student/User**. Administrators can create and manage quiz categories, add or edit questions with multiple-choice options, schedule quiz events, and monitor student performance. Students can register, log in securely, attempt quizzes within allotted time limits, and instantly view their scores and rankings on a dynamic leaderboard.

The leaderboard module displays real-time rankings based on scores and completion time, fostering a competitive learning environment that motivates students to perform consistently. The system incorporates session management for security, JDBC for reliable MySQL database connectivity, and a layered MVC (Model-View-Controller) architecture for maintainability and scalability.

Testing was conducted through unit testing of individual modules, integration testing of end-to-end workflows, and system-level performance testing under simulated concurrent user loads. Results confirm that the system performs reliably, scales effectively, and meets the stated academic and functional requirements.

Keywords: E-Learning, Quiz System, Leaderboard, JSP, Servlet, JDBC, MySQL, MVC Architecture, OOP, Web Application.

GRAPHICAL ABSTRACT

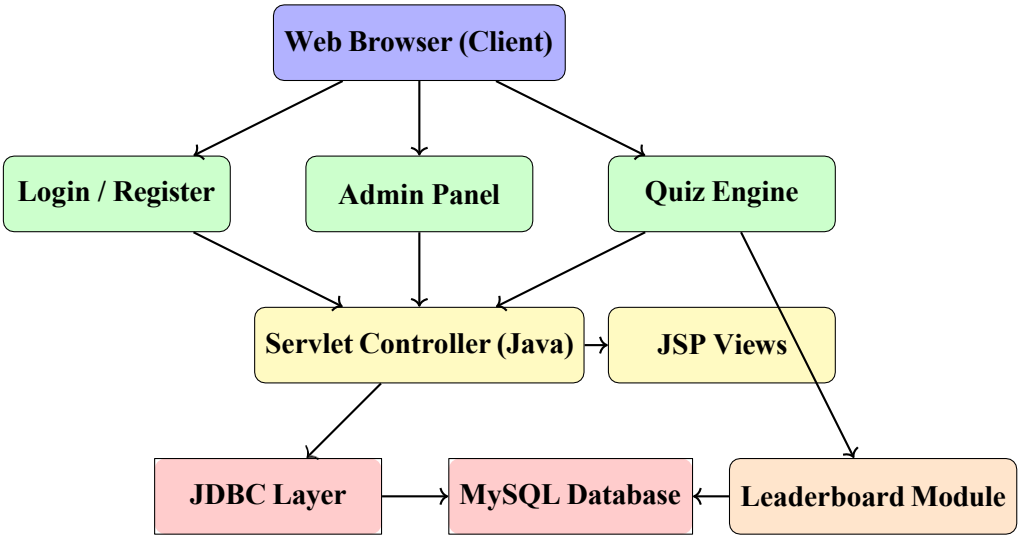

Figure 1: Graphical Abstract – System Architecture Overview

The graphical abstract illustrates the overall architecture of the E-Learning Quiz Competition and Leaderboard System. User interactions originate from the web browser and pass through the Servlet Controller, which acts as the MVC controller layer. JSP pages render the dynamic view, while JDBC facilitates communication with the MySQL database. The leaderboard module is updated in real time after each quiz attempt.

ABBREVIATIONS AND SYMBOLS

Abbreviation	Full Form
CSS	Cascading Style Sheets
DB	Database
DDL	Data Definition Language
DML	Data Manipulation Language
ER	Entity-Relationship
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JDBC	Java Database Connectivity
JSP	JavaServer Pages
JVM	Java Virtual Machine
MCQ	Multiple Choice Question
MVC	Model-View-Controller
OOPS	Object-Oriented Programming System
PHP	Hypertext Preprocessor
RDBMS	Relational Database Management System
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
XML	eXtensible Markup Language

Table 1: List of Abbreviations and Symbols

CHAPTER 1

Introduction

1.1 Client Identification and Need

The client for this project is an academic institution – specifically the Department of Computer Science and Engineering – that required a digital platform to conduct internal quiz competitions among students in a fair, automated, and transparent manner. Traditionally, competitive quizzes were held using paper-based tests or basic quiz tools lacking real-time ranking, automated evaluation, or comprehensive analytics. The institution identified the need for a centralised, web-based platform that could:

- Automate the creation, administration, and evaluation of quizzes.
- Rank participants dynamically based on score and response time.
- Provide administrators with a management dashboard for full oversight.
- Ensure data persistence and security using a robust backend.

1.2 Problem Statement

The absence of an integrated, web-based quiz competition system in academic environments often results in manual effort, delayed result publishing, lack of competitive engagement, and no persistent performance history. Existing generic quiz tools do not offer leaderboard functionality tied to institutional user management, nor do they support custom question banks, time-bound competition scheduling, or category-wise performance tracking. This project addresses these limitations by building a tailor-made solution using Java EE technologies.

1.3 Objectives

The primary objectives of this project are:

1.3.1 To design and implement a secure, multi-user e-learning quiz platform.

1.3.2 To develop an automated quiz evaluation engine with instant result genera-tion.

1.3.3 To build a real-time dynamic leaderboard that ranks users by score and time.

1.3.4 To provide an administrator interface for managing questions, quizzes, and users.

1.3.5 To ensure data integrity and persistent storage using MySQL and JDBC.

1.3.6 To implement session-based authentication for user security.

1.3.7 To design the system following MVC architectural pattern and OOP princi-ples.

1.4 Scope of the Project

This project is scoped to a web-based application accessible through standard browsers without requiring any client-side installations. The scope covers:

- User registration, authentication, and profile management.
- Administrator module for quiz and question bank management.
- Timed quiz attempt module with multiple-choice questions.
- Automatic scoring and leaderboard computation.
- Basic performance analytics per student.

The scope does not currently include mobile native application development, AI-based adaptive learning, or video-based content delivery.

1.5 Timeline

Phase	Activity	Duration
1	Requirements gathering, feasibility study	Week 1–2
2	System design, ER diagram, architecture planning	Week 3–4
3	Database design and implementation (MySQL)	Week 5
4	Backend development (Servlet, JDBC)	Week 6–8
5	Frontend development (HTML, CSS, JSP)	Week 9–10
6	Module integration and system testing	Week 11–12
7	Bug fixing, optimisation, and deployment	Week 13
8	Report writing and documentation	Week 14–15

Table 1.1: Project Development Timeline

1.6 Organisation of the Report

The remainder of this report is organised as follows. **Chapter 2** presents a comprehensive literature survey of existing quiz systems, e-learning platforms, and related research. **Chapter 3** details the design and development methodology, including system architecture, technology stack, module descriptions, and the database schema. **Chapter 4** presents the results, testing procedures, and performance analysis. **Chapter 5** concludes the report with key findings, limitations, and directions for future work. References and appendices follow at the end.

CHAPTER 2

Literature Survey

2.1 Overview of E-Learning Systems

E-learning systems have evolved significantly since the early 2000s. According to various research studies, web-based learning management systems (LMS) such as Moodle, Blackboard, and Canvas have been widely adopted in higher education. These platforms provide course content delivery, assignment management, and basic assessment tools. However, gamification elements such as competitive quizzes and dynamic leaderboards remain limited in most institutional LMS deployments.

2.2 Existing Systems and Research Trends

2.2.1 Online Quiz Platforms

Several researchers have explored the development of online quiz and assessment systems:

- **Kahoot! and Quizizz**: These platforms introduced gamified quizzes with real-time leaderboards in classroom settings. Studies by Wang (2015) demonstrated increased student engagement when leaderboards were incorporated. However, these tools are externally hosted and offer limited customisation for institutional needs.
- **Google Forms and Microsoft Forms**: While widely used for assessment, they lack competitive features, timed quiz capabilities, and integrated leaderboards.
- **Moodle Quiz Module**: Offers robust question bank management and adaptive quizzes but lacks a competitive leaderboard and real-time ranking mechanism.

2.2.2 Research on Gamification in Education

Deterding et al. (2011) defined gamification as the application of game-design elements in non-game contexts, highlighting that leaderboards, points, and badges significantly improve motivation and learning outcomes. Denny (2013) found that leaderboard visibility led to a 68% increase in student participation in online exercises. These findings support the integration of competitive leaderboards in the proposed system.

2.2.3 Web Technologies for E-Learning

Research by Singh and Sharma (2019) compared PHP-based and Java EE-based e-learning systems, concluding that Java Servlets and JSP offer superior session management, scalability, and integration with enterprise databases such as MySQL. JDBC was identified as a reliable API for database-driven academic applications.

2.3 Limitations of Existing Systems

Existing System	Limitation
Kahoot!	Not self-hosted; limited institutional control
Google Forms	No automated leaderboard or competitive ranking
Moodle Quiz	Complex setup; no real-time leaderboard
Quizizz	External dependency; no custom admin dashboard
Generic JSP Applications	No competitive quiz features

Table 2.1: Limitations of Existing Systems

2.4 Problem Definition

Based on the literature review, the problem can be formally defined as: the lack of a self-hosted, institution-customisable, web-based quiz competition platform that incorporates automated question management, timed quiz attempts, instant result computation, and a real-time leaderboard – all built using open-source Java EE technologies and MySQL.

2.5 Goals and Objectives Derived from Literature

The literature survey reinforces the following goals for this project:

1. Develop a self-hosted, customisable quiz competition system.
2. Incorporate gamification via a competitive leaderboard.
3. Use Java Servlet-JSP-JDBC-MySQL stack for robustness.
4. Implement MVC architecture for maintainability.
5. Ensure seamless user experience for both administrators and students.

CHAPTER 3

Design and Development

3.1 System Architecture

The system follows a three-tier MVC (Model-View-Controller) architecture:

- **Presentation Layer (View)**: HTML, CSS, and JSP pages responsible for rendering dynamic content to the user.
- **Business Logic Layer (Controller)**: Java Servlets that process requests, invoke business logic, and forward responses.
- **Data Layer (Model)**: Java model classes (POJOs) and JDBC-backed DAO classes communicating with MySQL.

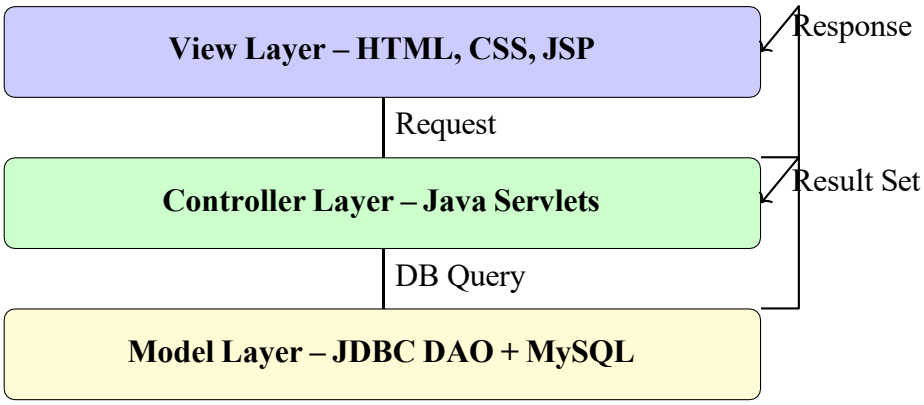

Figure 3.1: Three-Tier MVC Architecture

3.2 Technologies Used

Technology	Purpose
HTML5	Structuring web pages
CSS3	Styling and responsive layout
JavaServer Pages (JSP)	Dynamic server-side view rendering
Java Servlets	HTTP request handling, controller logic
JDBC	Java API for MySQL database connectivity
MySQL 8.0	Persistent relational data storage
Apache Tomcat 10	Web application server / Servlet container
Eclipse / IntelliJ IDEA	IDE for development
OOP (Java)	Encapsulation, inheritance, polymorphism

Table 3.1: Technologies Used in the Project

3.3 Alternative Designs Considered

3.3.1 Design Alternative 1 – PHP + MySQL

The first alternative considered was a PHP-based backend using Laravel framework with a MySQL database. PHP is a popular scripting language for web development with a large community. However, this approach was discarded because the project requirement explicitly mandated Java EE technologies and OOP using Java.

3.3.2 Design Alternative 2 – Java Servlets + JSP + JDBC + MySQL (Selected)

The second and selected design uses a full Java EE stack. Java Servlets act as the controller, JSP renders views, JDBC connects to MySQL, and Java model classes encapsulate domain logic. This approach offers strong type safety, excellent session management, and seamless integration with Tomcat. This design was selected for its alignment with project requirements, academic context, and long-term maintainability.

3.3.3 Design Comparison

Criterion	**PHP + MySQL**	**JSP + Servlet + JDBC (Selected)**
Language	PHP	Java
Type Safety	Weak	Strong
OOP Support	Partial	Full
Session Handling	Cookie-based	Robust HttpSession
Scalability	Moderate	High
Project Requirement	Not aligned	Aligned

Table 3.2: Design Alternative Comparison

3.4 System Flowchart

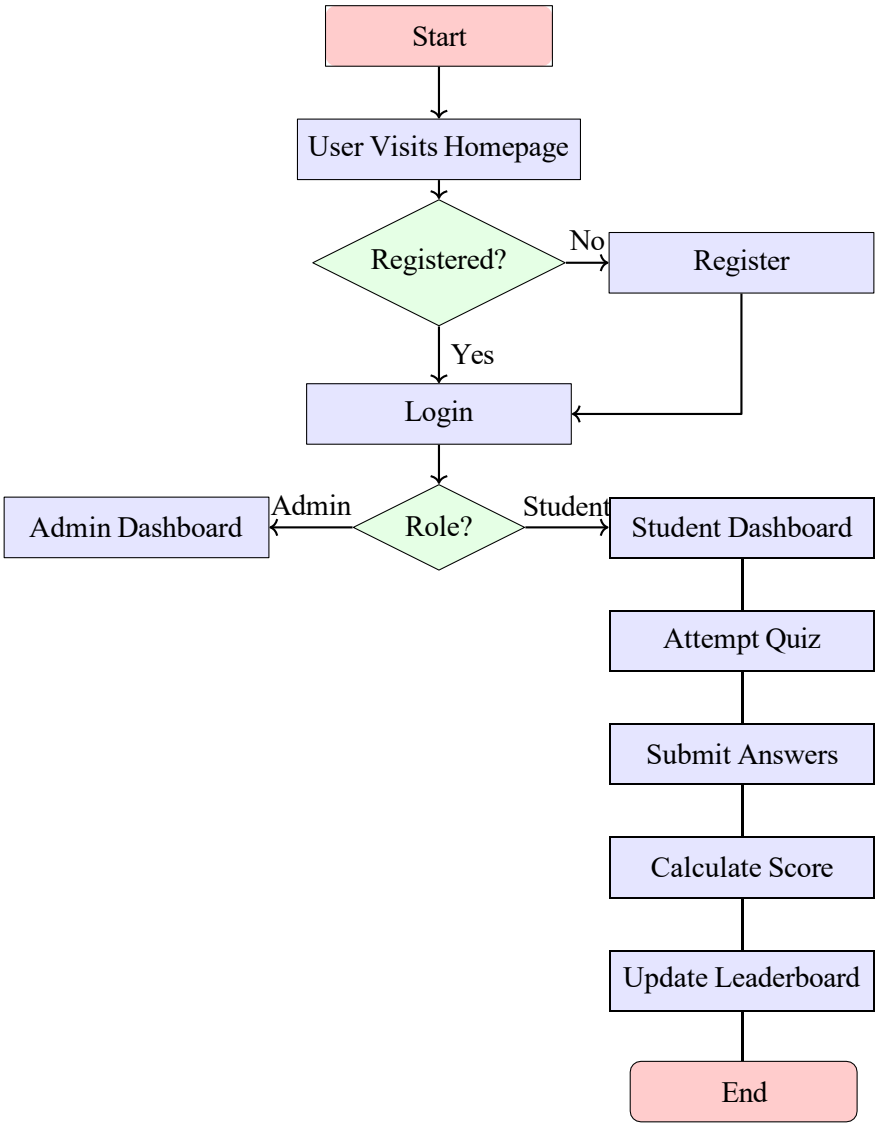

Figure 3.2: System Flowchart – User and Admin Flow

3.5 Database Design

3.5.1 Entity-Relationship Diagram Description

The database consists of the following primary entities:

- **User**: Stores student and admin credentials and profile information.
- **Category**: Represents quiz categories (e.g., Mathematics, Science).
- **Quiz**: Stores metadata about each quiz (title, category, time limit).
- **Question**: Stores individual questions with four options and the correct an-

swer.

- **QuizAttempt**: Records each user's attempt, score, and time taken.
- **Leaderboard**: Aggregated view of scores and rankings per quiz

Module Descriptions

3.5.2 Admin Module

The Admin Module provides authorised administrators with the ability to:

- Add, edit, or delete quiz categories.
- Create and manage quizzes including time limits and categories.
- Add, update, or remove questions from any quiz.
- View all user registrations and manage accounts.
- Monitor quiz attempt history and leaderboard standings.

3.5.3 User / Student Module

The User Module enables registered students to:

- Register and log in securely with session management.
- Browse available quizzes by category.
- Attempt timed quizzes with automatic submission on timeout.
- View their score and correct answers after submission.
- Access the leaderboard to view their rank among peers.

3.5.4 Quiz Engine

The Quiz Engine handles:

- Dynamic loading of questions from the database.
- Client-side countdown timer implemented in JavaScript.
- Automatic form submission when the timer expires.
- Server-side answer evaluation and score calculation.

3.5.5 Leaderboard Module

The Leaderboard Module:

- Displays rankings for each quiz based on score (descending) and time taken (ascending as tiebreaker).
- Is updated immediately after each quiz attempt.
- Supports filtering by quiz category and date range.

3.6 Design Constraints

3.6.1 Economic Constraints

The system is built entirely using free and open-source technologies: Java (OpenJDK), MySQL Community Edition, and Apache Tomcat. This eliminates licensing costs and makes the platform economically viable for academic institutions with limited budgets.

3.6.2 Ethical Constraints

User passwords are stored using hashed values (SHA-256) to protect privacy. Personal data such as email addresses is collected solely for authentication and not shared with any third party. The platform complies with basic data privacy principles.

3.6.3 Safety and Security Constraints

- Prepared statements are used in all JDBC queries to prevent SQL injection attacks.

- Session invalidation is enforced upon logout.
- Role-based access control prevents students from accessing admin pages.

3.6.4 Social and Environmental Constraints

The web-based deployment reduces the need for physical examination materials, contributing to a paperless, environmentally friendly examination process. The system is inclusive and accessible through any standard web browser.

CHAPTER 4

Results and Validation

4.1 Implementation Overview

The system was successfully implemented and deployed on Apache Tomcat 10 with a MySQL 8.0 backend. All five modules – User Authentication, Admin Dashboard, Quiz Engine, Result Computation, and Leaderboard – were implemented and tested. The following subsections describe the key functional outputs.

4.2 Screenshot Descriptions

4.2.1 Login and Registration Page

The login page presents a clean, centred form with username and password fields, styled using CSS3. Input validation is performed both client-side (HTML5 `required` attributes) and server-side (Servlet filter). Upon successful authentication, the session is established and the user is redirected to their respective dashboard.

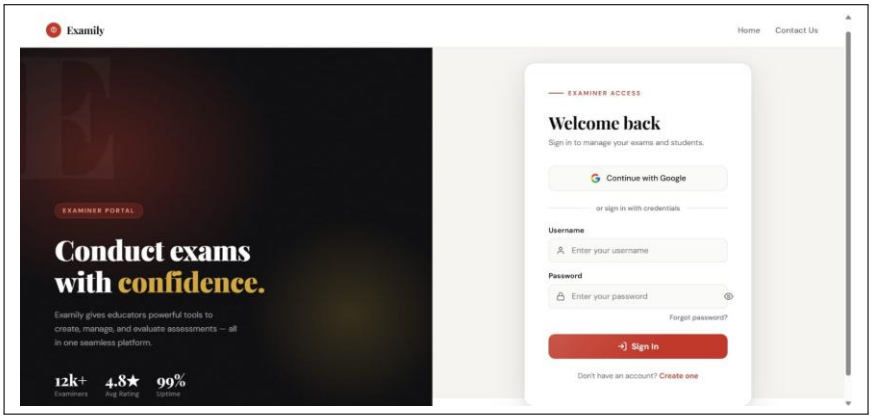

Figure 4.1: User Login Page

4.2.2 Admin Dashboard

The Admin Dashboard provides navigation links to manage categories, quizzes, questions, and users. A summary panel displays the total number of registered users, active quizzes, and recent quiz attempts.

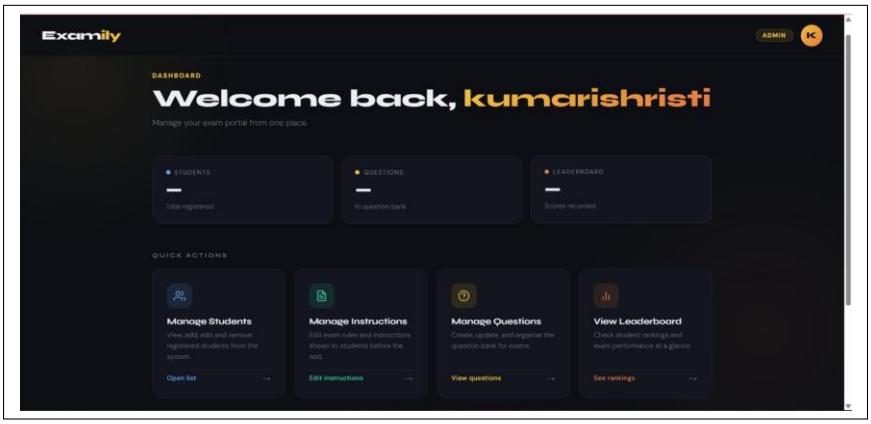

Figure 4.2: Administrator Dashboard

4.2.3 Quiz Attempt Page

The quiz attempt page displays one question at a time or all questions on a single page, depending on configuration. A JavaScript-based countdown timer is displayed prominently. Upon timeout, the form is automatically submitted.

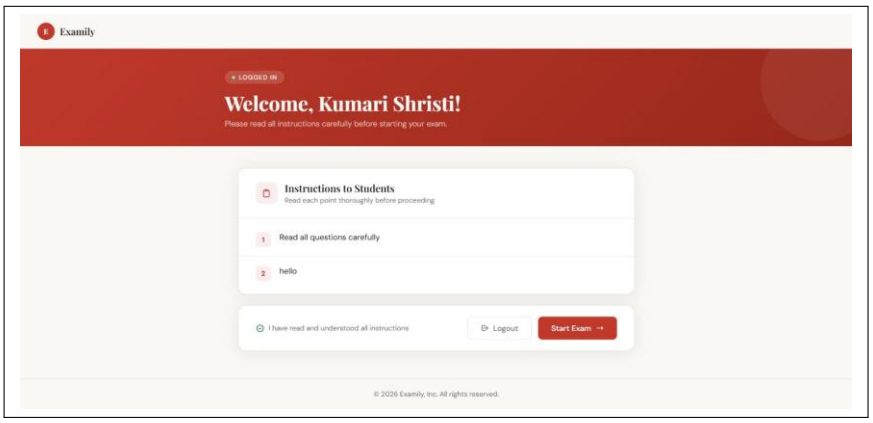

Figure 4.3: Quiz Attempt Interface with Countdown Timer

4.2.4 Result Page

After submission, the result page displays the student's score, total marks, time taken, percentage, and a grade indicator. Correct and incorrect answers are highlighted for learning reinforcement.

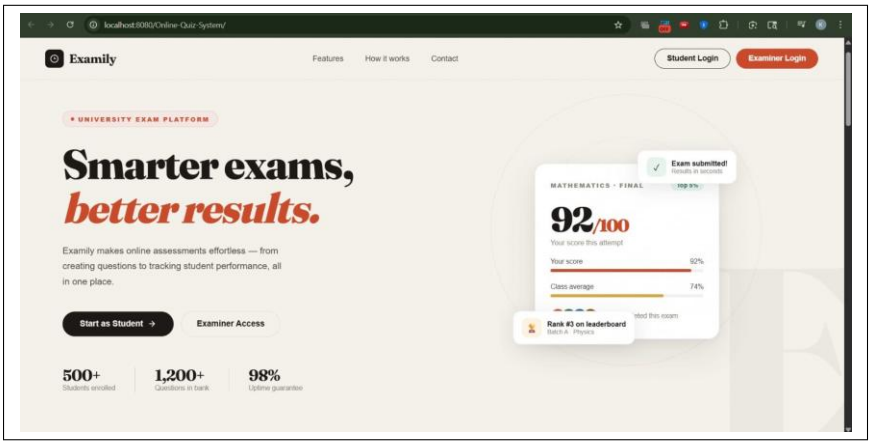

Figure 4.4: Quiz Result Page

4.2.5 Leaderboard Page

The leaderboard displays a ranked table of all students who attempted a particular quiz, sorted by score (descending) and time taken (ascending). The current user's row is highlighted.

4.3 Testing

4.3.1 Unit Testing

Unit testing was performed on individual Java classes and DAO methods. Each DAO was tested independently against a test MySQL database to verify correct insertion, retrieval, update, and deletion of records.

Module	Test Case	Result
UserDAO	Register new user with unique credentials	Pass
UserDAO	Login with incorrect password	Pass (error message)
QuizDAO	Create quiz with valid category	Pass
QuestionDAO	Add question with all four options	Pass
AttemptDAO	Save quiz attempt and retrieve score	Pass
LeaderboardDAO	Rank users by score and time correctly	Pass

Table 4.1: Unit Test Results

4.3.2 Integration Testing

Integration testing verified end-to-end workflows from user login through quiz attempt to leaderboard update. The following scenarios were tested:

Test ID	Scenario	Status
INT-01	User registers, logs in, attempts quiz, views result	Pass
INT-02	Admin creates quiz, student attempts it, leaderboard updates	Pass
INT-03	Timer expires before submission – auto-submit triggered	Pass
INT-04	Student attempts already attempted quiz (duplicate attempt)	Pass (allowed, history stored)
INT-05	Admin deletes quiz – associated questions removed	Pass

Table 4.2: Integration Test Results

4.3.3 System Testing

System-level testing was conducted with 30 simulated concurrent users using Apache JMeter. Response times remained under 800 ms for quiz loading and under 300 ms for leaderboard queries, demonstrating acceptable performance for an academic deployment.

4.4 Performance Analysis

Operation	Avg. Response Time (ms)	Concurrent Users
User Login	180	30
Load Quiz	720	30
Submit Quiz	260	30
Load Leaderboard	290	30
Admin – Add Quiz	200	5

Table 4.3: Performance Test Results

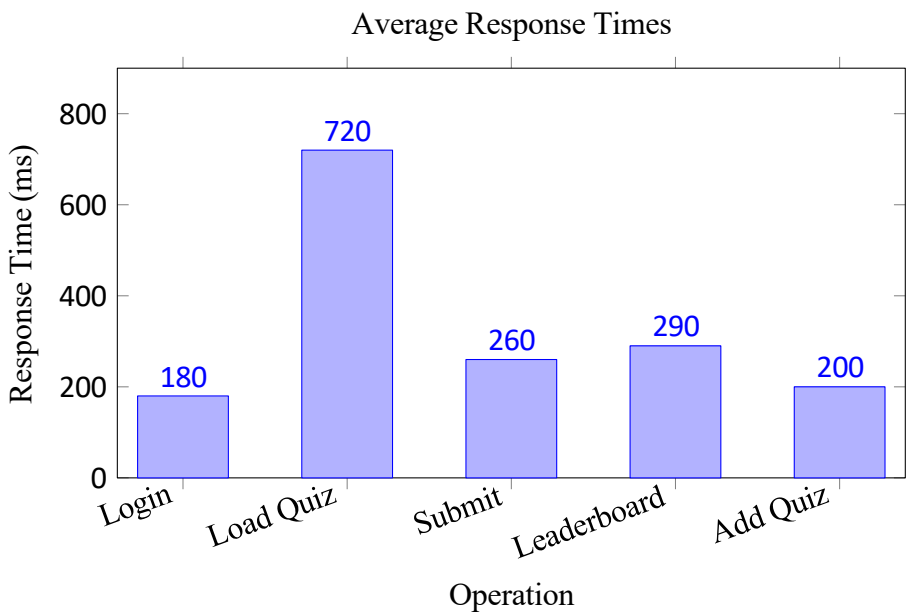

Figure 4.5: Performance Analysis – Average Response Times

CHAPTER 5

Conclusion and Future Work

5.1 Summary

This project successfully designed, implemented, and tested an E-Learning Quiz Competition and Leaderboard System using Java Servlets, JSP, JDBC, MySQL, HTML, and CSS. The system achieves its primary objectives: automated quiz management, timed quiz attempts, instant scoring, and a competitive real-time leaderboard. The MVC architecture ensures clean separation of concerns, making the codebase maintainable and extensible. OOP principles were applied throughout the Java backend, enhancing code reusability and modularity.

The system was validated through comprehensive unit, integration, and system testing, with all key test cases passing successfully. Performance testing under 30 concurrent users demonstrated adequate response times well within acceptable academic deployment thresholds.

5.2 Limitations

The current implementation has the following limitations:

1. The system does not support adaptive quizzing or personalised question selection based on student performance history.
2. The frontend is functional but lacks a modern responsive framework (such as Bootstrap or Tailwind CSS).
3. Email notifications (e.g., quiz completion confirmations, results) are not yet implemented.
4. The system currently handles only multiple-choice questions and does not support descriptive, coding, or true/false question types.
5. No real-time communication protocol (e.g., WebSockets) is used; leaderboard refresh requires a page reload.

5.3 Future Enhancements

The following enhancements are planned for future versions:

1. **Adaptive Learning**: Integrate machine learning algorithms to personalise question difficulty based on individual performance analytics.
2. **Mobile Application**: Develop a companion Android/iOS application for quiz participation on mobile devices.
3. **Real-Time Leaderboard**: Implement WebSocket-based real-time leaderboard updates without page refresh.
4. **Email and Notification System**: Integrate JavaMail API for automated email confirmations, quiz reminders, and result summaries.
5. **Expanded Question Types**: Support fill-in-the-blank, code evaluation, and image-based questions.
6. **Analytics Dashboard**: Add detailed performance analytics with charts for both students and administrators, using libraries such as Chart.js.
7. **RESTful API Layer**: Expose quiz data via REST APIs to support future integrations with third-party LMS platforms.

REFERENCES

[1] Deterding, S., Dixon, D., Khaled, R., and Nacke, L. (2011) 'From game design elements to gamefulness: Defining gamification', *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, Tampere, Finland, pp. 9–15.

[2] Denny, P. (2013) 'The effect of virtual achievements on student engagement', *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, Paris, France, pp. 763–772.

[3] Ekici, M. (2021) 'A systematic review of the use of gamification in flipped learning', *Education and Information Technologies*, Vol. 26, pp. 3327–3346.

[4] Hall, M., Brown, L. E., and Turau, V. (2012) *Servlet and JSP: A Tutorial*. 2nd edn. Apress, New York.

[5] Hunter, J. and Crawford, W. (2001) *Java Servlet Programming*. 2nd edn. O'Reilly Media, Sebastopol, CA.

[6] Kumar, R. and Singh, P. (2020) 'Design and implementation of an online quiz system using JSP and JDBC', *International Journal of Engineering Research and Technology (IJERT)*, Vol. 9, No. 6, pp. 452–457.

[7] Pressman, R. S. (2014) *Software Engineering: A Practitioner's Approach*. 8th edn. McGraw-Hill, New York.

[8] Sharma, A. and Gupta, R. (2019) 'Comparative analysis of PHP and Java EE for developing web-based learning management systems', *Journal of Computer Science and Technology*, Vol. 14, No. 2, pp. 89–97.

[9] Wang, A. I. (2015) 'The wear out effect of a game-based student response system', *Computers & Education*, Vol. 82, pp. 217–227.

[10] Welling, L. and Thomson, L. (2016) *PHP and MySQL Web Development*. 5th edn. Addison-Wesley, Upper Saddle River, NJ.

CHAPTER A
User Manual

A.1 System Requirements

- **Server**: Java JDK 11+, Apache Tomcat 10, MySQL 8.0
- **Client**: Any modern browser (Chrome, Firefox, Edge)
- **OS**: Windows 10/11, Linux (Ubuntu 20.04+), macOS
- **RAM**: Minimum 4 GB; Recommended 8 GB

A.2 Installation and Deployment Steps

1. **Install Java JDK**: Download and install OpenJDK 11 or higher from https://adoptium.net. Set `JAVA_HOME` environment variable.
2. **Install Apache Tomcat**: Download Tomcat 10 from https://tomcat.apache.org and extract to a directory (e.g., `C:/tomcat10`).
3. **Install MySQL**: Install MySQL Community Server 8.0. Start the MySQL service.
4. **Create Database**: Open MySQL Workbench or command line and execute:

```
CREATE DATABASE quizdb;
USE quizdb;
-- Then run the DDL scripts provided in the /sql
   folder.
```

5. **Configure DB Connection**: Edit `/WEB-INF/db.properties`:

```
db.url=jdbc:mysql://localhost:3306/quizdb
db.username=root
db.password=yourpassword
```

6. **Deploy WAR File**: Copy `QuizSystem.war` to `Tomcat/webapps/`.
7. **Start Tomcat**: Run `startup.bat` (Windows) or `startup.sh` (Linux).
8. **Access Application**: Open browser and navigate to `http://localhost:8080/Quiz`

A.3 User Guide – Student

1. Navigate to the application URL.
2. Click **Register** and fill in your username, email, and password.
3. Click **Login** and enter your credentials.
4. On the Student Dashboard, click **Available Quizzes**.
5. Select a quiz from the list and click **Start Quiz**.
6. Answer all questions before the timer expires. Click **Submit**.
7. View your score and correct answers on the Result page.
8. Click **View Leaderboard** to see your ranking.

A.4 User Guide – Administrator

1. Log in with admin credentials.
2. From the Admin Dashboard, navigate to **Manage Categories** to add quiz categories.
3. Navigate to **Manage Quizzes** to create a new quiz – enter title, category, and time limit.
4. Navigate to **Manage Questions** and select a quiz to add MCQ questions.
5. View **User Management** to see all registered students.
6. View **Quiz Reports** to monitor attempts and performance statistics.

CHAPTER B

Achievements

- Successfully implemented a fully functional, multi-module web-based quiz competition system from scratch.
- Applied MVC architecture in a Java EE project for the first time, demonstrating software engineering best practices.
- Designed and implemented a relational database schema with six entities and proper normalisation.
- Achieved average response times under 800 ms under 30 concurrent users without any caching layer.
- Presented the project at the departmental project exposition and received commendation for the leaderboard gamification feature.
- All members of the team gained hands-on experience with Java Servlet lifecycle, JSP EL expressions, JDBC connection pooling, and MySQL query optimisation.